Lesson 11: Wave After Wave

GameMaker

VIDEO GAME CREATION

This week: clearing a wave of enemies no longer means the game is over - it means the NEXT wave begins. Clear all 3, and that's when you really win.

Open your own [YourName]Game project from Lesson 10 - don't start a new one.

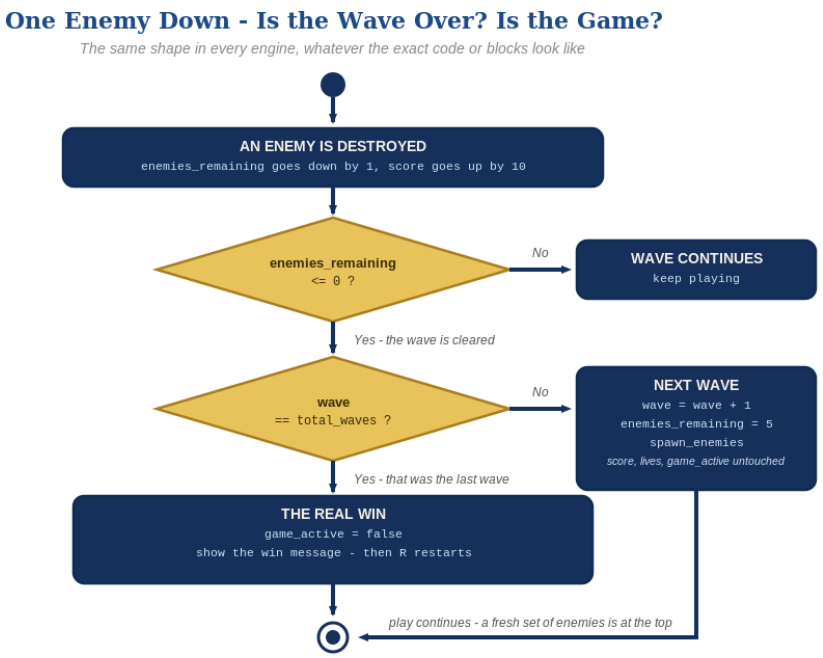
Tip: no new assets this week - waves reuse the spawn routine you already built in Lesson 10. This completes the Level 2 Complete milestone!

Setup

1. Open your [YourName]Game project from Lesson 10.
2. Create a new Script resource named scr_spawn_enemies. GameMaker lets you keep each reusable function in its own script, like scr_start_game from last week.

Level 1 - Minimal (MVP)

Clearing a wave's enemies spawns a fresh wave, up to 3 total. Clearing wave 3 triggers the real win. The current wave number is shown on screen.



Clearing a wave asks a second question. If it was not the last wave, the game spawns a fresh set of enemies and carries on. The same shape in every engine.

1. In scr_spawn_enemies, write a function spawn_enemies() holding just the loop that creates the five obj_enemy instances. Move that loop out of start_game() (code below). Nothing else belongs in this function.
2. In scr_start_game, replace the spawn loop inside start_game() with a call to spawn_enemies(), and add obj_gamemanager.wave = 1; to the resets.
3. On obj_gamemanager's Create event, add wave = 1; and total_waves = 3; alongside your other shared game state.
4. On obj_gamemanager's Draw GUI event, add a draw_text() call showing the current wave under Score and Lives.
5. On obj_bullet's Collision event with obj_enemy, revise the win check into an if/else (code below): if this was the last wave, win as before. Otherwise advance wave, reset enemies_remaining and call spawn_enemies() again.
6. Press F5 (Run) and play through. Clearing wave 1 or 2 should bring a fresh set of five instead of ending the game. Clearing wave 3 should show "You Win!".

Highlighted lines are new this week; everything else is code you already have.

```
// scr_spawn_enemies (new Script resource):
function spawn_enemies() {
    for (var enemy_count = 0; enemy_count < 5; enemy_count++) {
        instance_create_layer(80 + enemy_count * 96, 80, "Instances",
obj_enemy);
    }
}
```

```
// scr_start_game (revised - the spawn loop is gone):
function start_game() {
    with (obj_enemy) {
        instance_destroy();
    }
    obj_gamemanager.__dnd_score = 0;
    obj_gamemanager.__dnd_lives = 3;
    obj_gamemanager.wave = 1;
    obj_gamemanager.enemies_remaining = 5;
    obj_gamemanager.has_won = false;
    obj_gamemanager.has_lost = false;
    obj_gamemanager.game_active = true;
    spawn_enemies();
}

// obj_gamemanager's Create event:
enemies_remaining = 5;
has_won = false;
__dnd_lives = 3;
has_lost = false;
__dnd_score = 0;
game_active = true;
wave = 1;
total_waves = 3;

// obj_gamemanager's Draw GUI event:
draw_text(20, 20, "Score: " + string(__dnd_score));
draw_text(20, 50, "Lives: " + string(__dnd_lives));
draw_text(20, 80, "Wave: " + string(wave) + " of " + string(total_waves));
if (has_won) {
    draw_text(400, 50, "You Win! Press R to Restart");
}
if (has_lost) {
    draw_text(400, 90, "Game Over - Press R to Restart");
}

// obj_bullet's Collision event with obj_enemy:
audio_play_sound(sfx_hit, 1, false);
obj_gamemanager.enemies_remaining -= 1;
obj_gamemanager.__dnd_score += 10;
if (obj_gamemanager.enemies_remaining <= 0) {
    if (obj_gamemanager.wave >= obj_gamemanager.total_waves) {
        obj_gamemanager.has_won = true;
        obj_gamemanager.game_active = false;
    } else {
        obj_gamemanager.wave += 1;
        obj_gamemanager.enemies_remaining = 5;
        spawn_enemies();
    }
}
instance_destroy(other);
instance_destroy(self);
```

Alternate Path: GML Visual (no typing)

The same wave pattern works with drag-and-drop blocks. Every block that changes shared state needs its Target set to obj_gamemanager, exactly as in Lessons 8 to 10. The diagram below shows the finished stacks.

1. Create a new Script resource named scr_spawn_enemies. Its top-level block is "Declare a New Function" with Name = spawn_enemies, and everything below goes INSIDE that function block. Open scr_start_game and drag your existing spawn-loop blocks (the Repeat 5 block holding Create Instance, with its enemy_count blocks) out of the start_game function and into the new one. Check the connectors: a loose block runs at game start, before any room exists, and fails with a "layer does not exist" error.
2. Back in scr_start_game, inside the start_game function, add a "Set Variable" block after your existing Set blocks: Target obj_gamemanager, Variable wave, Value 1, Relative unchecked. Then, as the last block inside the function, add a "Function Call" block with Function = spawn_enemies (leave Target empty and Temp unchecked). Use Function Call, not Execute Script, which does not create any enemies.
3. On obj_gamemanager's Create event, add two "Set Variable" blocks (no Target needed, they run on obj_gamemanager itself): wave = 1 and total_waves = 3.
4. On obj_gamemanager's Draw GUI event, below your Score and Lives blocks, add a "Draw Value" block (Drawing category): Caption "Wave: ", Value wave, X 20, Y 80.
5. On obj_bullet's Collision event with obj_enemy, find your existing "If Variable" block that checks enemies_remaining (Target obj_gamemanager, less than or equal to 0). Inside it, add a second "If Variable" block: Target obj_gamemanager, Variable wave, operation greater than or equal to, Value obj_gamemanager.total_waves. Put your existing win blocks (Set Variable has_won = true and Set Variable game_active = false, both Target obj_gamemanager) inside this second block.
6. Directly after that second If Variable block add an "Else" block (Control category). Inside the Else: a "Set Variable" block (Target obj_gamemanager, Variable wave, Value 1, Relative checked); a "Set Variable" block (Target obj_gamemanager, Variable enemies_remaining, Value 5, Relative unchecked); and a "Function Call" block with Function = spawn_enemies.



scr_spawn_enemies, scr_start_game, obj_gamemanager and obj_bullet - the block-for-block equivalent of the Level 1 code.

Before You Test: Check Your Game State

Remember Lesson 10: a restart must put ALL game state back. This week you added new state, so check it too:

1. wave: confirm start_game() sets obj_gamemanager.wave = 1, so a restart always begins at wave 1. (In Visual, that is the new Set Variable block inside the start_game function.)
2. The Wave line is drawn from wave every frame, so it resets itself. Anything extra you added for waves (a message, a timer, an effect) needs its own reset inside start_game().

Level 2 - Complete (Alpha/Beta)

Wave transitions are seamless and correct every time - no double-spawns, no skipped waves, exactly 3 waves before the real win, and restarting always returns to wave 1. Nothing in the code needs to change for Level 2; it is about testing what you built.

- Play a full 3-wave game: confirm waves 1 and 2 each spawn a fresh set of five with no win message, and only wave 3 shows "You Win!".
- Let all 3 lives run out partway through wave 2 and confirm "Game Over" still appears, not a wave transition.
- Press R after a win or a loss and confirm wave is back to 1, along with score and lives, with exactly five enemies on screen.

Level 3 - Refined (Ready to Publish) - pick any that interest you

- The Stalemate is back - and now it can freeze the game at any of the three waves. If you fixed it in Lesson 10, check that your fix still works with waves. If you didn't, this may be the week it gets annoying enough to deal with. Find the cause and fix it in the way you, the Designer, think best. (Test it: let some enemies escape, destroy the rest, in wave 1, wave 2 and wave 3.)
- Visual: briefly flash a "Wave 2!" message in Draw GUI using a short-lived timer variable, separate from the has_won and has_lost text.
- Structure: notice spawn_enemies() is now called from two places (start_game() and the else branch of the bullet's win check), but the enemy-creating loop exists only once.

If Something's Wrong

- Wave 1 immediately shows "You Win!": confirm the if/else compares wave >= total_waves, and that the Else belongs to that inner If, not the outer one.
- New waves never spawn: confirm the else branch calls spawn_enemies(). In Visual, that must be a Function Call block, not Execute Script.
- "Layer does not exist" error at game start (Visual): the spawn-loop blocks are loose in scr_spawn_enemies instead of nested inside the Declare a New Function block.
- Restarting leaves wave at whatever it was: confirm start_game() sets obj_gamemanager.wave = 1.
- A wave never ends after you let some enemies escape - no next wave, no win, no Game Over: that is the Stalemate from Lesson 10 (Level 3), not a typo in your code. Escaped enemies cost a life but never count toward clearing the wave.

Want More?

Student Resource List > GameMaker section, or gamemaker.io/en/tutorials